

Text Normalization

Natural Language Processing

Ayesha Enayet

Corpus

- A **corpus** (plural: **corpora**) is a large collection of written or spoken texts that are used for linguistic analysis and natural language processing (NLP) tasks. Corpora are often used to study language patterns, perform text analysis, train machine learning models, and more. They can consist of written texts (e.g., books, articles, web pages) or spoken language (transcripts of conversations, speeches).

NLTK: Natural Language Toolkit

 IDLE Shell 3.12.5

File Edit Shell Debug Options Window Help

```
Python 3.12.5 (tags/v3.12.5:ff3bc82, Aug 6 2024, 20:45:27) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> import nltk
>>> from nltk.corpus import brown
>>> nltk.download('brown')
[nltk_data] Downloading package brown to
[nltk_data] C:\Users\ayasha.enayat\AppData\Roaming\nltk_data...
[nltk_data] Unzipping corpora\brown.zip.
True
```

```
>>> brown.categories()[0:10]
['adventure', 'belles_lettres', 'editorial', 'fiction', 'government', 'hobbies', 'humor', 'learned', 'lore', 'mystery']
```

```
>>> brown.raw()[0:1000]
"\n\n\tThe/at Fulton/np-tl County/nn-tl Grand/jj-tl Jury/nn-tl said/vbd Friday/nr an/at investigation/nn of
/in Atlanta's/np$ recent/jj primary/nn election/nn produced/vbd ``/`` no/at evidence/nn ''/' that/cs any/d
ti irregularities/nns took/vbd place/nn ./.\n\n\n\tThe/at jury/nn further/rbr said/vbd in/in term-end/nn pr
esentments/nns that/cs the/at City/nn-tl Executive/jj-tl Committee/nn-tl ,/, which/wdt had/hvd over-all/jj
charge/nn of/in the/at election/nn ,/, ``/`` deserves/vbz the/at praise/nn and/cc thanks/nns of/in the/at C
ity/nn-tl of/in-tl Atlanta/np-tl ''/' for/in the/at manner/nn in/in which/wdt the/at election/nn was/bedz
conducted/vbn ./.\n\n\n\tThe/at September-October/np term/nn jury/nn had/hvd been/ben charged/vbn by/in Ful
ton/np-tl Superior/jj-tl Court/nn-tl Judge/nn-tl Durwood/np Pye/np to/to investigate/vb reports/nns of/in p
ossible/jj ``/`` irregularities/nns ''/' in/in the/at hard-fought/jj primary/nn which/wdt was/bedz won/vbn
by/in Mayor-nominate/nn-tl Ivan/np Allen/np Jr./"
```

Text Normalization

- converting the text in to a more convenient, standard form.
 - Tokenization
 - separating out or tokenizing words from the text (basic definition)
 - Stemming
 - strip suffixes from the end of the word
 - Lemmatization
 - the task of determining that two words have the same root, despite their surface differences.
 - E.g. sang, sung, and sings
 - Sentence segmentation
 - breaking up a text into individual sentences, using cues like sentence segmentation periods or exclamation points.

Tokenization

- Design a Tokenizer for the following Sentence:
 - "Don't you love 🤖 Transformers? We sure do."
- Number of tokens?
- https://huggingface.co/docs/transformers/main/en/tokenizer_summary

Tokenization (types)

- Word-based

- ["Don't", "you", "love", "🤗", "Transformers?", "We", "sure", "do."] (space)
- ["Don", "'", "t", "you", "love", "🤗", "Transformers", "?", "We", "sure", "do", "."] (space + punctuation)
- ["Do", "n't", "you", "love", "🤗", "Transformers", "?", "We", "sure", "do", "."] (space + punctuation + rules)

- Character-based

- ["D", "o", "n", "'", "t", "y", "o", "u" ...]

- Subwords-based

- ["Don", "'", "t", "you", "love", "🤗", "Transform", "ers", "?", "We", "sure", "do", "."]

- Advantage of subword-based algorithm over word based?
 - Out-of-Vocabulary words!
 - Chinese and similar languages

Tokenization (Algorithms)

- **Wordpiece**
 - BERT
- Byte Pair Encoding (BPE) (Reading)
 - GPT
- Unigram

Wordpiece (Algorithm)

1. split by adding that prefix (#) to all the characters
 - w ##o ##r ##d
 2. learns merge rules
 - $\text{score} = (\text{freq_of_pair}) / (\text{freq_of_first_element} \times \text{freq_of_second_element})$
 - the algorithm prioritizes the merging of pairs where the individual parts are less frequent in the vocabulary.
 3. and we continue like this until we reach the desired vocabulary size
- [WordPiece tokenization - Hugging Face NLP Course](#)

- ("hug", 10), ("pug", 5), ("pun", 12), ("bun", 4), ("hugs", 5)
- ("h" "##u" "##g", 10), ("p" "##u" "##g", 5), ("p" "##u" "##n", 12), ("b" "##u" "##n", 4), ("h" "##u" "##g" "##s", 5)
- Vocabulary: ["b", "h", "p", "##g", "##n", "##s", "##u", "##gs"]
- Corpus: ("h" "##u" "##g", 10), ("p" "##u" "##g", 5), ("p" "##u" "##n", 12), ("b" "##u" "##n", 4), ("h" "##u" "##gs", 5)

- Vocabulary: ["b", "h", "p", "##g", "##n", "##s", "##u", "##gs", "hu"]
- Corpus: ("hu" "##g", 10), ("p" "##u" "##g", 5), ("p" "##u" "##n", 12), ("b" "##u" "##n", 4), ("hu" "##gs", 5)

- Vocabulary: ["b", "h", "p", "##g", "##n", "##s", "##u", "##gs", "hu", "hug"]
- Corpus: ("hug", 10), ("p" "##u" "##g", 5), ("p" "##u" "##n", 12), ("b" "##u" "##n", 4), ("hu" "##gs", 5)

Readings (Quiz)

- Briefly explain how Wordpiece algorithm is different from BPE?
- Identify at least one model that uses BPE algorithm for tokenization.

Byte-pair Encoding Algorithm

1. Normalization
2. Pre-tokenization
3. Splitting the words into individual characters
4. Applying the merge rules learned in order on those splits
 - At any step during the tokenizer training, the BPE algorithm will search for the most frequent pair of existing tokens.
 - That most frequent pair is the one that will be merged, and we rinse and repeat for the next step.

<https://huggingface.co/learn/nlp-course/en/chapter6/5>

NLTK (Tutorial)

- <https://pythonprogramming.net/stop-words-nltk-tutorial/?completed=/tokenizing-words-sentences-nltk-tutorial/>